

System Design - Terminologies

Roadmap:

1. *Foundation* - what, computation, components, functional and non functional aspects
2. *Communication* - DNS, API's, http, RESTful suite
3. *Data Layer* - Sql, no Sql, Cache
4. *Scaling and distribution* - CAP, load balancer, replication, partitioning
5. *Reliability and Operations* - Message queues, fault tolerance, Monitoring
6. *Case Studies* - in different notes.

Analogy to Cashier-Customer:

How can we improve this ?

- Improve skills of cashier.
- Suppose cashier has 3 tasks (money, receipt, checkout) - we must delegate these tasks to different people.
- Increase number of cashiers and their delegations as well - but in this database must be central
- balancing load on all the cash counters - equally spread in ideal case.

Mapping to system design:

- Improve code quality - TC and SC, maintainability, extensibility - LLD and DSA.
- Vertical Scaling - improving the single server that we have.
- Horizontal Scaling - adding more servers.
- Load balancers - delegating the tasks to each server properly.

Synchronous and Asynchronous calls:

if we wait for some service to complete and respond back - Synchronous call.

if we do not wait for some service to complete and respond back - Asynchronous call.

Message queues are used for handling asynchronous tasks.

Types of Applications: (Both have latency issue - what we have to fix)

1. *Data Intensive Apps* - less maths, more fetch calls from databases - optimisations in Disk I/O, Network calls, Memory.
2. *Compute Intensive Apps* - more maths, less data from databases - GPU bounds - optimisations in GPU/CPU.

Data-Intensive Application Characteristics



Continuous Data Flow

Large volumes of data are read and written continuously.



Network Calls

Multiple network calls are involved in serving requests.



Speed Focus

The focus is on delivering data from the database to the client as fast as possible.



Bottleneck Identification

Upgrading client-side CPU won't help — the bottleneck is in data access and transfer.

Solved by - caching, Sharding, replication, consistency, DB optimisations.

Compute-Intensive Application Characteristics



CPU/GPU Bound

Tasks are heavily reliant on CPU or GPU processing power.



Small Database Volume

The amount of data retrieved from the database is limited.



Heavy Processing

Significant processing is required before results are delivered.

eg - image processing, ML Model training, Cryptography, video rendering etc.
Solved by - Parallelism, GPU acceleration, algorithm optimisation

Types of Requirements:

1. Functional - user functions, features of app.
2. Non-Functional - uptime defined in SLA/SLO, scalability, fault tolerance, estimations of numbers, auto scaling, security, latency, authorisation, maintainability, reliability, observability.

Domain Name System:

We cannot store all the websites domain names map to IP addresses in our systems or browsers as the data is too large so we have DNS for resolving this issue.

Issues:

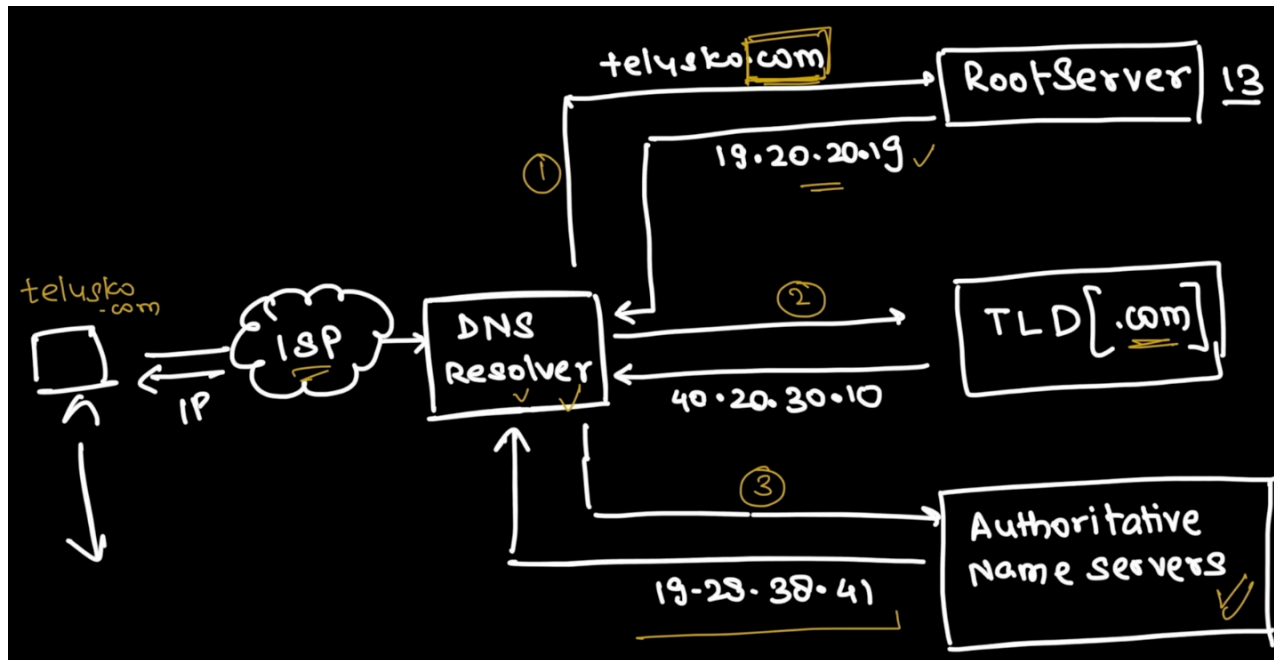
1. Storage
2. Modification of IP address maintenance.

DNS Resolution Process



Steps are as follows:

1. Your machine connects to the internet through an Internet Service Provider (ISP), which has its own DNS resolver.
2. There are 13 root servers worldwide, named a.root-server.net through m.root-server.net
3. The root server does not know the final IP address - it directs the resolver to the appropriate TLD server.
4. TLD are top level domains example - .com, .net, .in, .co.in etc.
5. The TLD server responds with the IP address of the Authoritative Name Server that manages the specific domain.
6. ANS holds the actual DNS records for the domain.
7. Returns the final IP address of the requested domain to the DNS Resolver.
8. The DNS Resolver returns the IP address to the browser.



DNS Resolution Process



User Input

The user types the website address into their browser (telusko.com)

The user's ISP's DNS resolver receives the request

DNS Resolver



Root Server

The DNS resolver queries the root server for the .com TLD server's IP address (a-m.root-server.net)

The root server returns the IP address of the .com TLD server

TLD Server



Authoritative Name Server

The TLD server returns the IP address of the authoritative name server (telusko.com)

The browser connects to the IP address of telusko.com, and the webpage loads

Browser Connection



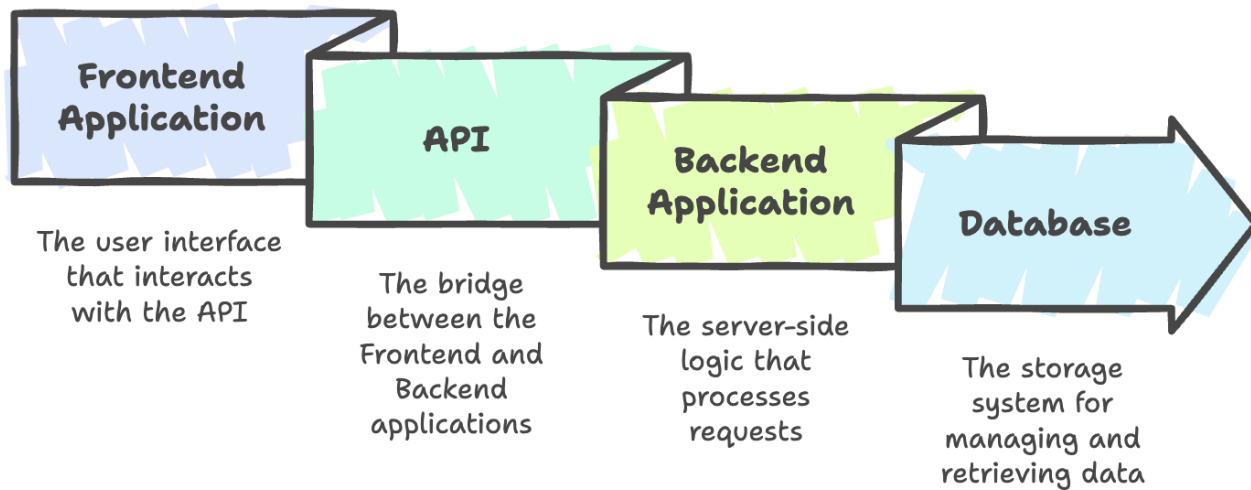
Caching in the whole process of DNS Resolution. There are zones for subdomain resolution in ANS part.

Cache Levels (checked in order)

Level	Location	Description
1	Browser Cache	Browser stores recently resolved domains locally
2	OS Cache	Operating system maintains its own DNS cache
3	DNS Resolver Cache	The ISP's resolver caches results for all users

API's - Application Programming Interfaces:

It is a set of endpoints exposed by an application that allows other applications (or frontends) to communicate with it, access its data, and perform operations without needing direct access to the underlying database.



If we give access to applications or frontends without APIs then it can be dangerous for our data in DB. So APIs restrict or control this access.

Types of APIs:

1. REST APIs - Representational State Transfer - data format JSON - uses HTTP methods.
2. SOAP APIs - Simple Object Access Protocols - data format XML - used for cross platform communication.
3. Graph QL - Graph Query Language - frontend can request multiple table's data in single query - data format JSON - solves issue of multiple requests (under-fetching) and over-fetching.
4. Web sockets - Real time - two way communication between client and server.
5. gRPC - google remote procedure call - data format protocol buffers - small in size so faster - micro service to micro service communication.

API Type	Data Format	Best For	Latency	Complexity
REST	JSON	General-purpose web APIs	Low	Simple
SOAP	XML	Legacy/enterprise systems	Moderate	High
GraphQL	JSON	Flexible frontend queries	Low	Moderate
gRPC	Protobuf	Microservice internal communication	Very Low	Moderate
WebSocket	Any	Real-time bidirectional communication	Real-time	Moderate

RESTful APIs:

These are standard format for transferring data from clients to servers, designing these depends upon resources.

endpoint = http method + path.

endpoints is location from which we can access on API.

Method - Defines the operation (GET, POST, PUT, PATCH, DELETE)

Path - The URL through which data is accessed or submitted

Requests in REST APIs:

Methods:

Method	Path	Purpose
GET	/users	Retrieve all users
GET	/users/{id}	Retrieve a specific user by ID
POST	/users	Create a new user (data sent in body)
PUT	/users/{id}	Replace the entire user record
PATCH	/users/{id}	Partially update specific fields
DELETE	/users/{id}	Delete a user

A REST API request consists of - method + path url + headers + body (optional)

headers - contains meta data, tokens, permissions for the request.

path url - tells api which resource to access.

Some more concepts:

1. Nesting - When entities have relationships. eg - /blogs/{id}/comments
2. Filtering - uses query parameters to narrow down results based on conditions. eg - /comments?post_id=123

There are three ways to send data in a REST API request:

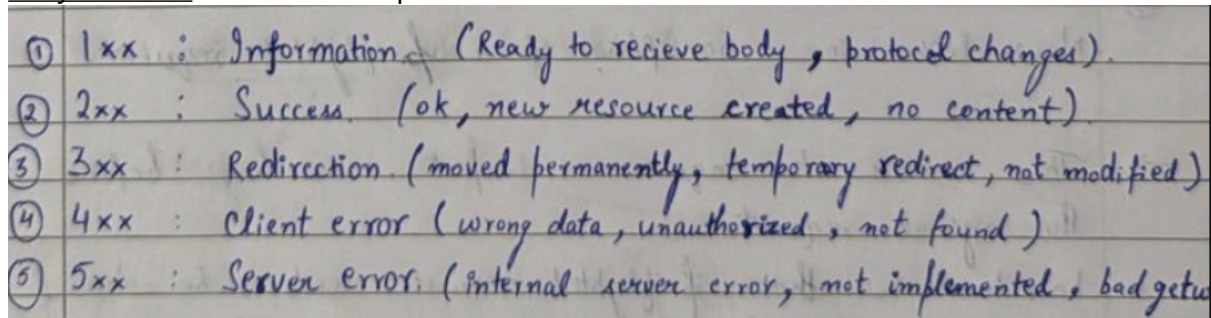
Method	Use Case	Visibility	Example
Path Parameter	Pass IDs, slugs, or unique identifiers	Exposed in URL	/users/42
Query Parameter	Filtering, sorting, pagination	Exposed in URL	/users?age=20&sort=name
Request Body	Sensitive or complex data (credentials, payloads)	Hidden from URL	{ "username": "muskan", "password": "***" }

Responses in REST APIs:

corresponds to what server sends for each client request.
consists of status code + body returned.

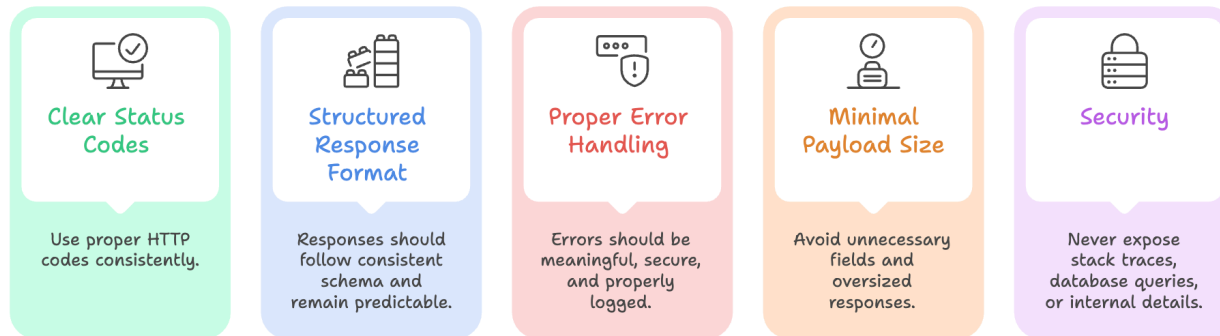
status codes - for real better understanding and documentation clarity.

body returned - the actual response from server.



①	1xx	: Information (Ready to receive body, protocol changes)
②	2xx	: Success (ok, new resource created, no content)
③	3xx	: Redirection (moved permanently, temporary redirect, not modified)
④	4xx	: Client error (wrong data, unauthorized, not found)
⑤	5xx	: Server error (internal server error, not implemented, bad getuo)

Characteristics of Good API Responses



Databases:

do it from DBMS notes for detailed explanations.

Caching:

What is cache?

Any memory/data that lets us avoid computation is known as cache.
Without caching - load on server increases, more latency to users.
Generally stored as Key-Value pairs.

These can be at certain places:
frontend - client side cache
backend - server side cache
database - WAL or DB cache

Some other terms:
cache hit - when the requested data is already present in the cache.
cache miss - when the requested data is not available in the cache.
TTL - Time to live (the time for which an entry will be auto deleted from cache)

Strategies of keeping updated cache:

1. Read Through Cache (RTC) - read from cache, write to DB and in case of cache miss out to cache.
2. Write Through Cache (WTC) - read from cache, write in cache and cache writes to DB. eg - stock prices
3. Write Around Cache (WAC) - read from cache, write always to DB. eg - Instagram posts.
4. Write Back Cache (WBC) - write to cache but write to DB is asynchronous so performance > consistency.

Strategy Comparison

Strategy	 Read-Through	 Write-Through	 Write-Around	 Write-Back
Write Target	Database	Cache → Database (sync)	Database	Cache → Database (async)
Read Target	Cache	Cache	Cache	Cache
Consistency	Moderate	Strong	Low (risk of stale data)	Eventual
Best For	Read-heavy workloads	Data requiring latest values	Infrequently read after write	High write traffic, speed-critical

Cache Eviction Policies:

1. Least Recently Used
2. Most Recently Used
3. Least Frequently Used
4. First in First out
5. Last in First out

Cache Eviction Policies					
Policy	 LRU	 MRU	 LFU	 FIFO	 LIFO
Evicts	Least recently accessed	Most recently accessed	Least frequently accessed	Oldest inserted entry	Newest inserted entry
Based On	Last access time	Last access time	Total access count	Insertion order	Insertion order
Best For	General-purpose caching	Data unlikely to be reused	Workloads with clear popularity	Simple, predictable eviction	Specific use cases

Load Balancer:

It is a component that delegates the incoming client's requests to the most appropriate server base on algorithms to improve fault tolerance, performance, scalability and availability.

Tasks:

1. Delegation - routing to server in most efficient way.
2. Health Checks - should know which server has ability at present.